

## OSSP PRACTICAL \_ 4

**Task-1** Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions

```
GNU nano 7.2                                pra4task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid[i] == 0) {
            printf("Child %d started. PID = %d\n", i + 1, getpid());

            sleep(2 + i);

            printf("Child %d completed. PID = %d\n", i + 1, getpid());
            exit(10 + i);
        }
    }

    // wait() waits for any child process
    printf("\nParent is waiting using wait()...\n");

    pid_t completed = wait(&status);

    if (completed > 0) {
        printf("wait(): Child PID %d completed\n", completed);

        exit(10 + i);
    }

    // wait() waits for any child process
    printf("\nParent is waiting using wait()...\n");

    pid_t completed = wait(&status);

    if (completed > 0) {
        printf("wait(): Child PID %d completed\n", completed);

        if (WIFEXITED(status))
            printf("Exit status = %d\n", WEXITSTATUS(status));
    }

    // waitpid() waits for a specific child
    printf("\nParent is waiting using waitpid()...\n");

    pid_t result = waitpid(pid[0], &status, 0);

    if (result > 0) {
        printf("waitpid(): Child PID %d completed\n", result);

        if (WIFEXITED(status))
            printf("Exit status = %d\n", WEXITSTATUS(status));
    }

    // Wait for remaining children
    while (wait(NULL) > 0);

    printf("\nAll child processes have completed.\n");
    printf("Parent process terminating. PID = %d\n", getpid());

    return 0;
}
```

```

ruhi0404@Ruhitha:~$ cd "/Operating_Systems"
ruhi0404@Ruhitha:~/Operating_Systems$ nano pra4task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc pra4task1.c -o pra4task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./pra4task1
Parent process started. PID = 628
Child 1 started. PID = 629
Child 2 started. PID = 630

Parent is waiting using wait()...
Child 3 started. PID = 631
Child 1 completed. PID = 629
wait(): Child PID 629 completed
Exit status = 10

Parent is waiting using waitpid()...
Child 2 completed. PID = 630
Child 3 completed. PID = 631

All child processes have completed.
Parent process terminating. PID = 628
ruhi0404@Ruhitha:~/Operating_Systems$ ps -f
UID          PID     PPID  C  STIME TTY          TIME CMD
ruhi0404      405       404  0   02:59 pts/0        00:00:00 -bash
ruhi0404      634       405  0   03:04 pts/0        00:00:00 ps -f
ruhi0404@Ruhitha:~/Operating_Systems$ top
top - 03:04:52 up 5 min,  1 user,  load average: 0.06, 0.05, 0.01
Tasks:  22 total,   1 running,  21 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem : 15853.6 total, 14097.3 free,   562.7 used,  1390.9 buff/cache
MiB Swap:  4096.0 total,  4096.0 free,    0.0 used. 15290.9 avail Mem

```

## Observation

- The parent process successfully created three child processes using `fork()`.
- Each child process executed independently and terminated after a different delay.
- The `wait()` function waited for any one child process to complete and returned the PID of the completed child.
- The `waitpid()` function waited for a specific child process using its PID.
- The parent process synchronized with all child processes before terminating.
- Thus, `wait()` provides general child-process synchronization, while `waitpid()` provides more specific control over which child process is waited for.

**Task-2 Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.**

```
GNU nano 7.2                                                                    pra4task2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    } else {
        printf("Parent process started. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping. Child will become a zombie.\n");

        sleep(30);

        printf("Parent process terminating.\n");
    }

    return 0;
}
```

```
ruhi0404@Ruhitha:~/Operating_Systems$ nano pra4task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc pra4task2.c -o pra4task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./pra4task2
Parent process started. PID = 718
Child PID = 719
Parent sleeping. Child will become a zombie.
Child process started. PID = 719
Child process exiting...

Parent process terminating.
ruhi0404@Ruhitha:~/Operating_Systems$ ps -el
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0    1    0  0  80   0  -  5450  -    ?         00:00:03 systemd
5 S   0    2    1  0  80   0  -  795   -    hvc0       00:00:00 init-systemd(Ub
0 S   0    7    2  0  80   0  -  795   -    hvc0       00:00:00 init
4 S   0   46    1  0  79  -1 - 12592  -    ?         00:00:00 systemd-journal
4 S   0   99    1  0  80   0  -  6288  -    ?         00:00:01 systemd-udev
4 S  991  114    1  0  80   0  -  5336  -    ?         00:00:00 systemd-resolve
4 S  996  115    1  0  80   0  -  22759 -    ?         00:00:00 systemd-timesyn
4 S   0   173   1  0  80   0  -  1061  -    ?         00:00:00 cron
4 S  101  174    1  0  80   0  -  2409  -    ?         00:00:00 dbus-daemon
4 S   0   197   1  0  80   0  -  4495  -    ?         00:00:00 systemd-logind
4 S  102  238    1  0  80   0  -  55629 -    ?         00:00:00 rsyslogd
4 S   0   242    1  0  80   0  -  781   -    tty1      00:00:00 agetty
4 S   0   267    1  0  80   0  -  26759 -    ?         00:00:00 unattended-upgr
5 S   0   403    2  0  80   0  -  796   -    ?         00:00:00 SessionLeader
5 S   0   404   403  0  80   0  -  800   -    ?         00:00:00 Relay(405)
4 S 1000  405   404  0  80   0  -  1884  do_wai pts/0    00:00:00 bash
```

## Observation

- A child process was created using `fork()`.
- The child terminated while the parent continued executing without calling `wait()` or `waitpid()`.
- The terminated child remained in the process table as a zombie (Z) / `<defunct>` process because its exit status had not been collected by the parent.
- The program was then modified to use `waitpid()` for proper synchronization.
- After the modification, the parent collected the child's exit status immediately after its termination.
- The child was therefore removed from the process table and no zombie process was created.
- This demonstrates that proper parent-child synchronization using `wait()` or `waitpid()` prevents zombie processes.